1. Title Page

Lab Name: Lab 13: Wireless LANs (WLAN) - WiFi and 802.11 Analysis **Date:** YYYY-MM-DD **Name:** [Your Name] **Lab Partners:** [Partners' Names, if any] **Software/Hardware Used:** Cisco Modeling Labs (CML) - Personal / Free Edition

---

2. Background

Wireless Local Area Networks (WLANs) operate under the **IEEE 802.11 (WiFi)** standard. Key concepts include:

1. **IEEE 802.11 Frame Format**: Unlike Ethernet frames which use a 2-address format (Source and Destination), 802.11 frames use up to a **4-address format**. This is because frames are transmitted over a shared wireless medium to an intermediate **Access Point (AP)**, which acts as a bridge to the wired distribution system.
2. **Addressing Mapping**: The roles of the address fields (Address 1, Address 2, Address 3, Address 4) are determined by the `To DS` (To Distribution System) and `From DS` flags in the Frame Control field:
   - `To DS = 0, From DS = 1` (Downlink from AP to Client): Address 1 is Destination (Client), Address 2 is BSSID (AP), Address 3 is Source (Sender).
   - `To DS = 1, From DS = 0` (Uplink from Client to AP): Address 1 is BSSID (AP), Address 2 is Source (Client), Address 3 is Destination (Receiver).
3. **802.11 Frame Types**: Frames are classified into:
   - **Management Frames**: Beacons, probe requests/responses, association requests/responses. (Beacons are broadcast periodically by the AP to advertise the network SSID, rates, and capabilities).
   - **Control Frames**: RTS/CTS (Request/Clear to Send) to avoid collisions and solve the hidden terminal problem, and ACKs to confirm frame delivery.
   - **Data Frames**: Encapsulate the actual network layer payloads.
4. **WPA2 4-Way Handshake**: Wireless links are encrypted to prevent eavesdropping. When a client associates, it performs a **4-way cryptographic handshake** using EAPOL (EAP over LAN) key exchanges to verify the Pre-Shared Key (PSK) and generate temporal encryption keys (PTK and GTK).

In this lab, you will deploy a wireless network in CML using a WiFi Access Point and two WiFi Client nodes, perform packet captures on the wireless interface, and analyze management, control, data, and security handshake sequences.

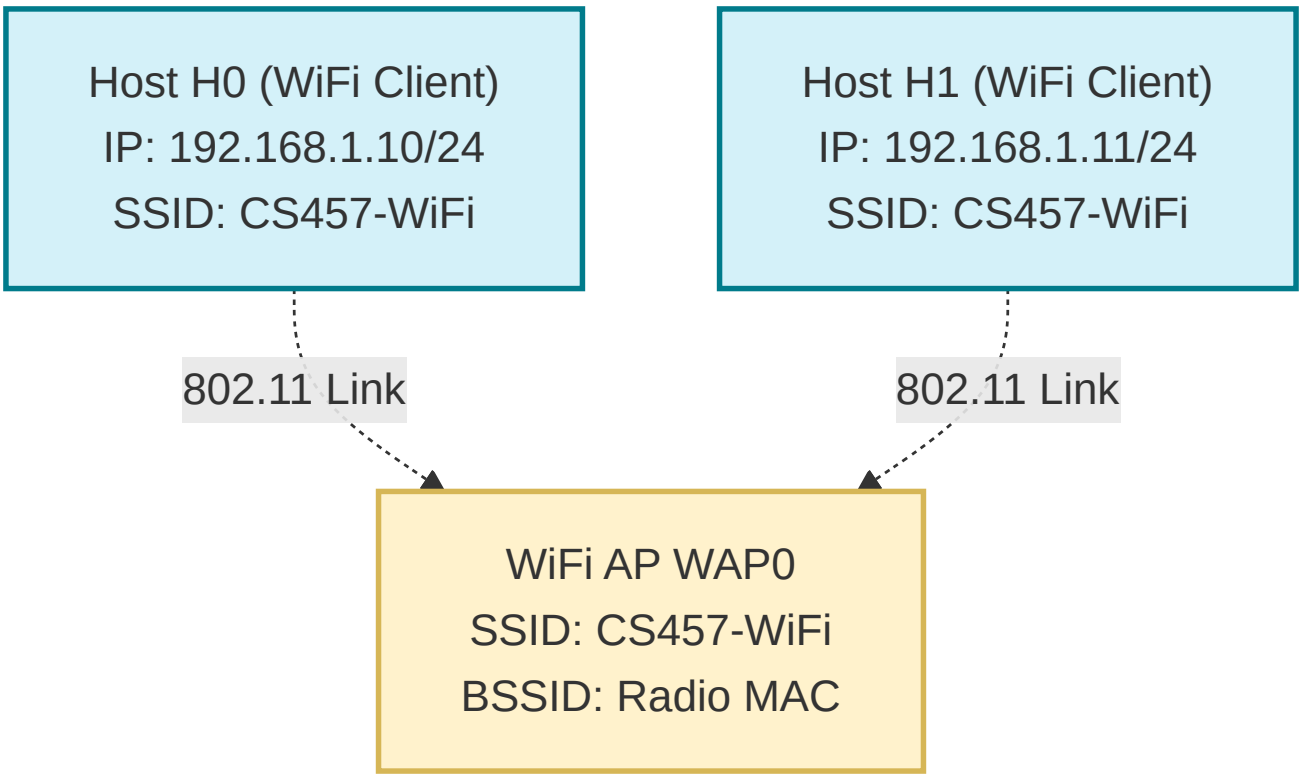
---

3. Objective / Purpose

To configure a basic WLAN infrastructure in CML, capture and analyze 802.11 management frames (beacons), inspect 802.11 frame address fields and control flag maps, and analyze the WPA2 4-way cryptographic key exchange in Wireshark.

---

4. Topology Diagram



Details:

- 1x WiFi Access Point (`<initials>-WAP0`) broadcasting SSID `CS457-WiFi` .
- 2x WiFi Clients (`<initials>-H0` and `<initials>-H1`) associated with the AP.

---

5. Equipment & Environment

- **Software:** Cisco Modeling Labs (CML), Terminal Emulator (e.g., PuTTY, SecureCRT, native terminal), Wireshark
- **Hardware / Virtual Nodes:** 2x WiFi Client Nodes, 1x WiFi Access Point Node

---

6. Configuration / Methodology

WLAN Configuration Table

Device	CML Node Name	Interface	IP Address	SSID	Security Mode	Pre-Shared Key
Access Point	`<initials>-WAP0`	Radio0	192.168.1.1/24	`CS457-WiFi`	WPA2-PSK (AES)	`wirelesspass123`
Client 0	`<initials>-H0`	wlan0	192.168.1.10/24	`CS457-WiFi`	WPA2-PSK (AES)	`wirelesspass123`
Client 1	`<initials>-H1`	wlan0	192.168.1.11/24	`CS457-WiFi`	WPA2-PSK (AES)	`wirelesspass123`

Step-by-Step Configuration Guide

Foreground vs. Background Daemon Execution

When configuring wireless nodes in Linux, wireless daemons (`hostapd` on APs and `wpa_supplicant` on Clients) can be executed in two modes:

- **Background Mode (`-B` flag)**: Runs the daemon silently in the background, freeing up your terminal prompt.
  - Start AP in background: `hostapd -B /home/cisco/hostapd.conf`
  - Start Client in background: `wpa_supplicant -B -i wlan0 -c /tmp/wpa_supplicant.conf`
- **Foreground Debug Mode (Omit `-B` , add `-d` or `-dd`)**: Prints real-time association events, probe requests, and EAPOL key handshakes directly to your terminal. Useful for troubleshooting.
  - Run AP in foreground debug mode: `pkill hostapd && hostapd -dd /home/cisco/hostapd.conf`
  - Run Client in foreground debug mode: `pkill wpa_supplicant && wpa_supplicant -dd -i wlan0 -c /tmp/wpa_supplicant.conf`
  - *(Press `Ctrl+C` to terminate a foreground daemon).*

---

1. Access Point Setup (`<initials>-WAP0`)

In CML, open the **Edit Config** tab of the WiFi AP node (`KH-WAP0`) and paste the following `#cloud-config` :

```
#cloud-config
hostname: KH-WAP0
manage_etc_hosts: True
system_info:
  default_user:
    name: cisco
password: cisco
chpasswd:
  expire: False
ssh_pwauth: True
ssh_authorized_keys:
  - your-ssh-pubkey-line-goes-here
write_files:
  - path: /home/cisco/hostapd.conf
    content: |
      interface=wlan0
      driver=nl80211
      ssid=CS457-WiFi
      channel=6
      auth_algs=1
      wpa=2
      wpa_key_mgmt=WPA-PSK
      wpa_passphrase=wirelesspass123
      rsn_pairwise=CCMP
      beacon_int=1000
    owner: cisco:cisco
    permissions: '0644'
runcmd:
  - ip addr add 192.168.1.1/24 dev wlan0
```

```
- ip link set wlan0 up
- hostapd -B /home/cisco/hostapd.conf
```

2. Client Host Setup (`<initials>-H0` and `<initials>-H1`)

Standard Linux nodes do not support WPA2-PSK via legacy `iwconfig` (which only supports WEP). To connect to a WPA2-PSK network, use `wpa_supplicant` with `p2p_disabled=1` and `psk="wirelesspass123"` :

Option A: Interactive CLI on Client Nodes

1. **Configure Host `<initials>-H0`** :

```
# Bring up interface and assign IP
ip addr add 192.168.1.10/24 dev wlan0
ip link set wlan0 up

# Create WPA2 PSK configuration file directly with ctrl_interface enabled for wpa_cli
cat << 'EOF' > /tmp/wpa_supplicant.conf
ctrl_interface=/var/run/wpa_supplicant
p2p_disabled=1
network={
    ssid="CS457-WiFi"
    psk="wirelesspass123"
    key_mgmt=WPA-PSK
}
EOF

# Kill any previous instance and start wpa_supplicant daemon
pkill wpa_supplicant
wpa_supplicant -B -i wlan0 -c /tmp/wpa_supplicant.conf
```

2. **Configure Host `<initials>-H1`** :

```
# Bring up interface and assign IP
ip addr add 192.168.1.11/24 dev wlan0
ip link set wlan0 up

# Create WPA2 PSK configuration file directly
cat << 'EOF' > /tmp/wpa_supplicant.conf
ctrl_interface=/var/run/wpa_supplicant
p2p_disabled=1
network={
    ssid="CS457-WiFi"
    psk="wirelesspass123"
    key_mgmt=WPA-PSK
}
EOF

# Kill any previous instance and start wpa_supplicant daemon
pkill wpa_supplicant
wpa_supplicant -B -i wlan0 -c /tmp/wpa_supplicant.conf
```

Option B: Client `#cloud-config` (in CML Edit Config tab for Clients)

```
#cloud-config
hostname: KH-H0
manage_etc_hosts: True
system_info:
  default_user:
    name: cisco
password: cisco
chpasswd:
  expire: False
ssh_pwauth: True
write_files:
  - path: /etc/wpa_supplicant/wpa_supplicant.conf
    content: |
      ctrl_interface=/var/run/wpa_supplicant
      p2p_disabled=1
      network={
          ssid="CS457-WiFi"
          psk="wirelesspass123"
          key_mgmt=WPA-PSK
      }
    owner: root:root
    permissions: '0644'
runcmd:
  - ip addr add 192.168.1.10/24 dev wlan0
  - ip link set wlan0 up
  - wpa_supplicant -B -i wlan0 -c /etc/wpa_supplicant/wpa_supplicant.conf
```

(For Host H1, adjust hostname to `KH-H1` and IP to `192.168.1.11/24`).

Step-by-Step Traffic Generation & Execution:

1. **Start Packet Capture in CML GUI**:
   - Right-click the Access Point (`<initials>-WAP0`) or Client (`<initials>-H0`) node.
   - Select **Packet Capture**, choose **`Wireless`** (which captures on the `wlan0` interface), and click **Start**. *(Note: Selecting **Wireless** ensures raw 802.11 frames—beacons, 4-address headers, and WPA2 EAPOL handshakes—are recorded).*
2. **Verify Client Association**: Open Host H0's console and verify connection status using native built-in tools (`iw` or `wpa_cli`):

   ```
   # Method 1: Using iw (standard modern netlink tool):
   iw dev wlan0 link

   # Method 2: Using wpa_cli status (from wpa_supplicant):
   wpa_cli status
   ```

   (Confirm that `SSID: CS457-WiFi` and the Access Point MAC address/BSSID are displayed, indicating successful association).

3. **Generate Traffic**: Ping Client H1 from Client H0:

```
ping -c 4 192.168.1.11
```

4. **Trigger & Log 4-Way Security Handshake**: To capture the complete WPA2 4-Way Handshake log output directly on the Access Point console: a. On AP node (`<initials>-WAP0`), stop background daemon and start `hostapd` in foreground debug mode:

```
pkill hostapd
hostapd -dd /home/cisco/hostapd.conf
```

   b. On Client H1 console, trigger re-association:

```
wpa_cli -i wlan0 reassociate
```

   c. Observe the live log stream on `<initials>-WAP0` displaying:

   - `sending 1/4 msg of 4-Way Handshake`
   - `received EAPOL-Key frame (2/4 Pairwise)`
   - `sending 3/4 msg of 4-Way Handshake`
   - `received EAPOL-Key frame (4/4 Pairwise)`
   - `EAPOL-4WAY-HS-COMPLETED` d. Take a screenshot of this terminal window for your lab submission.

5. **Download Capture File from CML**:

   - In the CML interface **Packet Capture** tab, click **Stop**.
   - Click **Download** to save the `.pcap` file directly to your local computer with one click, then open it in Wireshark.

---

7. Wireshark Analysis and Questions

Open your downloaded packet capture in Wireshark and answer the following questions:

Part A: 802.11 Management Frames (Beacons)

1. Locate a **Beacon frame** sent by the AP (`<initials>-WAP0`).
   - What is the Source MAC address (the transmitter)?
   - What is the BSSID of the wireless network? Does this match the AP's MAC?
2. Inspect the **Beacon parameters** in the packet details pane:
   - What SSID is advertised?
   - What WPA/WPA2 security requirements (e.g. Robust Secure Network - RSN group/pairwise cipher suites) are listed in the parameters?

Part B: 802.11 Frame Addressing & Control Flags

3. Locate an **802.11 Data frame** containing your ping traffic (ICMP request) sent from Client H0 to Client H1.
   - Expand the **Frame Control** field in the IEEE 802.11 header. What are the binary/hex values of the `To DS` and `From DS` flags?
   - Based on these flag states, why are these values set (what is the directional path of the packet)?
4. Inspect the address fields in the header of this data frame:

- What is the MAC address value for:
    1. Address 1 (Receiver Address / RA)
    2. Address 2 (Transmitter Address / TA)
    3. Address 3 (Source/Destination Address depending on DS flags)
- Map these addresses to the actual CML nodes in your topology (H0, H1, or WAP0).

5. Explain why an IEEE 802.11 wireless frame header requires up to **four** MAC address fields, whereas a standard Ethernet II wired frame header only requires **two**. (Discuss the bridging role of the AP).

Part C: WPA2 Security and 4-Way Handshake Analysis

6. Inspect the `hostapd` foreground console debug log output on Access Point `<initials>-WAP0` captured during Host H1's reconnection:
    - Identify the log messages corresponding to the 4 steps of the **4-Way Handshake** (`sending 1/4 msg` , `2/4 Pairwise` , `sending 3/4 msg` , `4/4 Pairwise`). How many total EAPOL key messages are exchanged?
    - What type of keys (PTK, GTK, PMK) are derived and installed during this handshake sequence as shown in the log output?
    - Why is the 4-Way Handshake necessary (what does it protect against in a wireless environment)?

---

8. Troubleshooting & Diagnostic Logging Commands

Real-Time Wireless Logging & Debugging Reference

1. Verbose Daemon Logging (`wpa_supplicant` & `hostapd`)

Run daemons in the foreground with extra debug flags (`-d` or `-dd`) to watch associations, probe requests, and 4-way handshakes live:

```
# On Client Nodes (H0 / H1) - Verbose Client Debugging:
pkill wpa_supplicant
wpa_supplicant -dd -i wlan0 -c /tmp/wpa_supplicant.conf

# Log client debug output to a file:
wpa_supplicant -dd -i wlan0 -c /tmp/wpa_supplicant.conf -f /tmp/wpa_debug.log &

# On Access Point (WAP0) - Verbose AP Debugging:
pkill hostapd
hostapd -dd /home/cisco/hostapd.conf
```

2. Real-Time Wireless Event Monitoring (`iw event`)

Monitor raw `nl80211` wireless events (auth, assoc, disassoc, connect, scan results) in real time:

```
# Monitor all kernel netlink wireless events live with timestamps:
iw event -t

# View associated station statistics (signal dBm, RX/TX bytes/packets):
iw dev wlan0 station dump
```

3. Real-Time Interactive CLI Debugging (`wpa_cli`)

Dynamically alter log levels or attach to the event log stream interactively:

```
# Increase log level dynamically without restarting daemon:
wpa_cli -i wlan0 log_level debug

# Open interactive wpa_cli monitoring shell:
wpa_cli -i wlan0
> level 0        # Shows all events (EAPOL, scan, state changes) live in terminal
> status         # Check current state detail
```

4. Live Console Packet Capture (`tcpdump`)

Inspect link-layer headers and EAPOL security frames directly on the Linux console:

```
# Capture and log 802.11/Ethernet link-layer headers live:
tcpdump -i wlan0 -e -n -v

# Filter specifically for EAPOL 4-Way Handshake frames (EtherType 0x888e):
tcpdump -i wlan0 -e -n -v 'ether proto 0x888e'

# Monitor ICMP ping traffic live:
tcpdump -i wlan0 -n icmp
```

5. Interface Errors & Kernel Logs (`dmesg` / `ip`)

```
# Check kernel wireless driver events:
dmesg | grep -E "wlan|mac80211|cfg80211|ath|iwl"

# Continuously follow kernel log output:
dmesg -w

# View detailed interface stats, dropped packets, and errors:
ip -s link show wlan0
```

Student Troubleshooting Notes

Issue Encountered: [Describe what went wrong, if anything]
Discovery Method: [How did you find the issue?]
Resolution: [How did you fix it?]

(If no issues were encountered, explain potential pitfalls for this configuration).

---

9. Conclusion & Analysis

Summarize your findings. Contrast the operations of 802.11 wireless networks with standard 802.3 wired Ethernet, detailing the role of management frames in network advertising, 3/4-address mapping over bridged access points, and the necessity of EAPOL handshakes for link-layer security.

---

10. What to Hand In

- The Lab YAML configuration file with your name, date, and name of the lab at the top in comments.
- A single PDF report containing:
  - Your written answers to the 6 Wireshark analysis questions in Section 7.
  - A screenshot of your active CML topology showing the running hosts (`<initials>-H0` , `<initials>-H1`) and AP (`<initials>-WAP0`).
  - A screenshot of Host H0's console showing the output of `iw dev wlan0 link` or `wpa_cli status` , confirming association.
  - A screenshot of the AP (`<initials>-WAP0`) console showing the output of `hostapd` running in foreground debug mode (`hostapd -dd /home/cisco/hostapd.conf`), confirming the full WPA2 4-Way Handshake sequence (`sending 1/4 msg` , `received EAPOL-Key frame (2/4 Pairwise)` , `sending 3/4 msg` , `received EAPOL-Key frame (4/4 Pairwise)` , and `EAPOL-4WAY-HS-COMPLETED`).